

PRODUKTÜBERSICHT UND SICHERHEITSARCHITEKTUR

ODRE PQC v0.2.9

Application-Embedded Post-Quantum Security Boundary

Integration · Sicherheitsgrenze · Betrieb

DEUTSCHE AUSGABE

September 2026

Dokumentstruktur

01	1. Produktübersicht
02	2. Minimale Integration
03	3. Produktarchitektur
04	4. Fail-Closed Security Contract
05	5. Platform Adapter und Native Runtime
06	6. Betriebsdiagnose
07	7. Worker- und Lifecycle-Architektur
08	8. Trust Boundaries und Attack Surface
09	9. Deployment-Architektur
10	10. Produktidentität
11	11. Fazit

Produktübersicht und Sicherheitsarchitektur

Application-Embedded Post-Quantum Security Boundary

ODRE PQC ist ein plattformübergreifendes Sicherheitsprodukt, das vor ausgewählten Anwendungsrouten eine überprüfbare Schutzgrenze einrichtet. Entwickler müssen die Details der Post-Quanten-Kryptografie nicht selbst zusammensetzen. Die öffentliche Integration bleibt klein; intern verbindet das Produkt Native-Runtime-Vertrauen, kryptografischen Handshake, Session, strikte Sequenz, Replay-Abwehr, Request-Semantik, geschützte Responses, Lifecycle-Kontrolle und Fail-Closed-Verhalten.

1. Produktübersicht

1.1 Definition

ODRE PQC ist keine Bibliothek, die lediglich PQC-Funktionen bereitstellt. Es ist eine Application Security Boundary zwischen geschützten FastAPI-Routen und einer nativen PQC-Runtime. Bevor ein geschützter Request den Business Handler erreicht, prüft das Produkt Runtime Identity, Protocol State, Session, Sequence und die authentifizierte Bedeutung des Requests.

Der Produktumfang umfasst:

- Identität der installierten Runtime und Artifacts
- ML-KEM-768 Key Encapsulation
- ML-DSA-65 Digital Signature
- Hybrid Handshake und Session-Aufbau
- Ablauf und Entsorgung von Sessions
- Strict Sequence und Replay Blocking
- Semantische Bindung von Method, Path, Query, Header und Body
- Bindung geschützter Response-Metadaten
- Fail-Closed-Abbruch vor Handler-Ausführung
- Worker-, Crash-, Restart- und Shutdown-Lifecycle
- Windows- und Ubuntu-spezifisches Native Trust
- Diagnose über [doctor](#), [verify](#) und [status](#)

1.2 Das zu lösende Problem

Ein PQC-Verfahren aufrufen zu können bedeutet nicht, einen Anwendungsrequest sicher verarbeiten zu können. Ein einsatzfähiges Produkt muss zusätzlich feststellen, welche Runtime geladen wurde, ob geprüftes und geladenes Objekt identisch sind, wie ein Handshake an eine Session gebunden ist, ob ein Replay den Handler zweimal ausführt, ob HTTP-Metadaten die Business-Semantik verändern und ob nach einem Crash veralteter Zustand weiterlebt.

ODRE PQC macht diese Fragen zu Bedingungen, die vor jeder Ausführung eines geschützten Handlers erfüllt sein müssen.

1.3 Zentrale Produktprinzipien

- Einfache Integration: Entwickler implementieren die kryptografische State Machine nicht selbst.
- Ende-zu-Ende gebundene Bedeutung: Geschützt wird, was die Anwendung tatsächlich verarbeitet, nicht nur ein verschlüsselter Body.
- Keine Ausführung ohne Nachweis: Geschützte Routen erlauben keinen stillen Bypass und keinen automatischen Classical-only Downgrade.

1.4 Zielumgebungen

ODRE PQC richtet sich an FastAPI-Dienste, geschützte APIs mit Anforderungen an Vertraulichkeit, Integrität und Replay-Abwehr, gemischte Windows-/Ubuntu-Flotten und Umgebungen mit CLI-basierter Betriebsdiagnose.

Element	Verifizierte Kombination
Windows	Windows Server 2022 AMD64 / CPython 3.12.10
Linux	Ubuntu 24.04.4 LTS AMD64 / CPython 3.12.3
Kryptografische Runtime	OpenSSL 3.5.8
Key Encapsulation	ML-KEM-768
Digitale Signatur	ML-DSA-65
Wire Protocol	odre-pqc/3

1.5 Verhältnis zu TLS, VPN, WAF und PQC Libraries

Komponente	Hauptrolle	Unterschied zu ODRE PQC
TLS	Kanal-Vertraulichkeit und -Integrität	ODRE PQC bindet Route, Session, Sequence und Request-Bedeutung.
VPN	Begrenzung der Netzwerkerreichbarkeit	ODRE PQC prüft geschützte Requests auch innerhalb erlaubter Netze.
WAF	HTTP-Pattern- und Policy-Filter	ODRE PQC kontrolliert die Ausführung des kryptografisch gültigen Requests.
PQC Library	Kryptografische Primitive	ODRE PQC fügt Primitive zu einem Fail-Closed-Lifecycle zusammen.
ODRE PQC	Eingebettete Anwendungsgrenze	Ein Vertrag von Runtime Trust bis Response Binding.

Diese Komponenten ergänzen sich. ODRE PQC fügt oberhalb von Transport-, Netzwerk- und HTTP-Filtern Bedingungen für die tatsächliche Anwendungsausführung hinzu.

1.6 Öffentliche und geschützte Routen

Öffentliche Routen können gemäß Anwendungspolitik verfügbar bleiben. Geschützte Routen werden erst nach erfolgreicher Prüfung von Runtime, Handshake, Session, Sequence und Request-Semantik ausgeführt. Kann die Schutz-Runtime keinen sicheren Zustand beweisen, bleiben öffentliche Health- oder Informationsrouten möglich, während geschützte Routen Fail-Closed reagieren: Fail-Closed with Public Availability.

2. Minimale Integration

2.1 Installationsablauf

```
Wheel installieren
→ odre-pqc init
→ odre-pqc doctor
→ odre-pqc verify
→ install(app)
→ Anwendung starten
→ odre-pqc status
```

Stufe	Zweck
Installation	Python Package und CLI Entry Points bereitstellen.
init	Plattform-Runtime und lokalen Sicherheitszustand initialisieren.
doctor	Umgebung, Rechte, Dateien und Runtime-Voraussetzungen diagnostizieren.
verify	Reale PQC Self-tests und Installationsintegrität prüfen.
install(app)	Security Boundary in FastAPI registrieren.
status	Runtime-Zustand nur lesend anzeigen.

2.2 FastAPI-Integrationspunkt

```
from fastapi import FastAPI
from odre_pqc import install

app = FastAPI()
install(app)
```

Die kleine Oberfläche trennt Verantwortlichkeiten. Application Handler verwalten weder ML-KEM Ciphertext noch ML-DSA Signatures, Replay Windows, Provider Paths oder Native Handles. Die Boundary erzwingt für jeden geschützten Request dieselbe Sicherheitsreihenfolge.

2.3 Verantwortung der Anwendung

Die Anwendung wählt geschützte Routen, prüft Benutzer- und Organisationsrechte, validiert Business Input, definiert Datenhaltung und entscheidet über öffentliche Funktionen. ODRE PQC stellt sicher, dass ein geschützter Request den Security Contract passiert hat, bevor diese Policies ausgeführt werden.

3. Produktarchitektur

3.1 Schichtenmodell

FastAPI Application

- ODRE PQC Integration Boundary
- Admission and Semantic Binding
- Handshake and Session Engine
- Sequence and Replay Gate
- Protected Handler

Integration Boundary → Platform Adapter → Native PQC Runtime

Die Architektur besteht aus Application Integration, Admission, Protocol, Semantic Binding, Platform Adapter und Native Runtime. Betriebssystemunterschiede bleiben unterhalb des gemeinsamen Python Security Contract.

3.2 Pipeline eines geschützten Requests

Protected Request

- Admission
- Handshake oder gültige Session
- Strict next-sequence
- Request Semantic Binding
- Protected Handler genau einmal
- Response Binding und Protection

Die Reihenfolge ist sicherheitsrelevant. Unsupported Streaming, unbounded Output Contracts, malformed Envelopes und oversized Inputs müssen vor möglichen Handler-Side-Effects abgewiesen werden.

3.3 Admission

Admission prüft Message Type, Pflichtfelder, Encoding, Gesamtgröße, widersprüchliche Felder, unterstützte Response-Formen und notwendige Security Metadata. Ein erfolgreicher Admission-Schritt bedeutet nur, dass die nächste Schicht den Request sicher interpretieren kann; er ist noch nicht authentifiziert.

3.4 Handshake und Session

Client Material

- Runtime Identity und Self-test
- ML-KEM-768 / ML-DSA-65
- Transcript, Path und Version binden
- Server Material und Signature
- Key Confirmation
- Session erstellen, Sequence = 0

Protocol Version, Algorithm Set, Materialien beider Seiten, Transcript, Route Meaning und Session Identifier werden gemeinsam gebunden. Kann eine notwendige Beziehung nicht verifiziert werden, entsteht keine Session.

3.5 Kryptografische Rollen

- ML-KEM-768: Kapselung und Wiedergewinnung gemeinsamen Secret Materials.
- ML-DSA-65: Authentisierung des Handshake Materials und Integrität des Transcripts.
- Hybrid Handshake: Bindet erforderliche Komponenten an denselben Peer, dieselbe Session und dieselbe Route.

Erfolgreiche Primitive allein stellen keinen Schutz her. Runtime Identity, Transcript Binding, Key Confirmation und Session-Aufbau müssen ebenfalls abgeschlossen sein. Das Scheitern einer Pflichtkomponente wird nicht zu Classical-only Success.

3.6 Session Lifecycle

Uninitialized → Handshaking → Active → Expired → Closed

↘ Validation Failure ↗

Eine Active Session ist nicht unbegrenzt vertrauenswürdig. Expiry, Runtime State Change, Worker Ownership Change, Sequence Violation oder Integrity Failure schließen den geschützten Zustand.

3.7 Strict Sequence und Replay Defense

Geschützte Requests benötigen exakt den nächsten Sequence-Wert. Duplicate, Rollback und Gap werden abgewiesen; bei konkurrierenden identischen Werten wird nur einer akzeptiert. Accept Decision und Sequence Update sind gegenüber Handler Dispatch atomar, damit derselbe Request den Business Handler nicht zweimal ausführt.

3.8 Request Semantic Binding

Nur den Body zu verschlüsseln lässt Method, Path, Query und relevante Header austauschbar. ODRE PQC bindet HTTP Method, normalisierten Path und Query, Security-relevante Header, Payload oder Envelope, Session ID, Sequence, Protocol Version und Message Type. Stimmen authentifizierte und Handler-sichtbare Bedeutung nicht überein, wird nicht ausgeführt.

3.9 Response Binding

Status Code, Content Type, geschützte Header, Cookies, Response Body sowie zugehörige Request Session und Sequence werden in das geschützte Ergebnis eingebunden. Nicht sicher darstellbare Response-Formen werden explizit abgewiesen.

4. Fail-Closed Security Contract

Fail-Closed bedeutet: Ein Request, dessen Schutzbedingungen nicht bewiesen werden können, erreicht den Protected Handler nicht. Öffentliche Routen oder der gesamte Prozess müssen dadurch nicht zwingend stoppen.

Geschützte Routen werden bei nicht beweisbarer Runtime Identity, Provider-/Dependency-Trust, PQC Self-test, Handshake Transcript, Session, Strict Sequence, Semantic Binding, Worker Ownership oder Recovery blockiert. Automatic Classical Downgrade gilt nicht als gleichwertiger Schutz.

Eine Ablehnung nach bereits ausgeführten Datenbank- oder Service-Side-Effects ist zu spät. Admission und Contract Decisions liegen daher möglichst vor dem Handler.

5. Platform Adapter und Native Runtime

5.1 Plattformgrenze

Common Python Security Contract

└─ Windows: DLL, ACL, Junction, Reparse, Loaded Module

└─ Ubuntu: SO, Permission, Symlink, FD, Inode, Loader

↓

Verified Native Runtime

5.2 Windows Native Trust

Windows Trust umfasst DLL-/Provider-Pfade, ACL und Ownership, Junction-/Reparse-Bypass, Dependency Search Paths, Verified-versus-loaded Identity, Runtime Replacement und Handle-/Module-Lifetime. Ein korrekter Pre-load Hash reicht nicht, wenn der Loader danach ein anderes Objekt oder eine andere Dependency wählen kann.

5.3 Ubuntu Native Trust

Ubuntu Trust umfasst Shared Object und Parent Permissions, Symlink und Unsafe Parent, ([st_dev](#), [st_ino](#)) Identity, Atomic Replacement zwischen Verify und Load, Provider-/Module-/Engine-/Config-Injection, Dependency Resolution und Recovery nach SIGKILL oder Restart. FD und Inode binden kritische Übergänge an reale Objekte statt an veränderliche Pfadstrings.

5.4 Runtime Archive und Manifest

Geprüft werden Platform Selection, Archive Hash, Per-file Manifest Hashes, Permissions und Parent Trust, Loaded-object Identity und Package-/Runtime-Kompatibilität. Die Initialisierung führt über Staging, Object Identity Lock, Atomic Promotion, Native Load, Version Check und reale ML-KEM-/ML-DSA-Self-tests. Fehlende Primitive oder Identity Proof führen zu Fail-Closed.

6. Betriebsdiagnose

- **init**: bereitet Runtime und lokalen Sicherheitszustand vor und erkennt konkurrierende, partielle oder nicht vertrauenswürdige Initialisierung.
- **doctor**: diagnostiziert OS, Architecture, Python, Runtime Identity, Permissions, Hashes, Symbols, Provider, Dependencies und State Store.
- **verify**: prüft Versionen, Native Load, reale ML-KEM-768/ML-DSA-65 Self-tests, Hybrid Handshake, Session Lifecycle, Protected Round Trip und fehlenden Automatic Downgrade.
- **status**: meldet den Zustand read-only, ohne Sessions oder Leases zu verändern oder Secrets offenzulegen.

7. Worker- und Lifecycle-Architektur

Konkurrierende Worker dürfen Runtime oder Sicherheitszustand nicht still in einem nicht unterstützten Ownership-Modell teilen. Normaler Shutdown stoppt neue Protected Admissions, drainiert begrenzte In-flight Requests, entsorgt Sessions und Leases, schließt Native Resources und löst Locks in dieser Reihenfolge.

Crash, SIGKILL und Stromverlust können sich nicht auf Cleanup verlassen. Beim Restart werden Process Ownership, Stale Locks, Sessions, Worker Leases, Status State und teilweise initialisierte Runtime geprüft.

```
Starting → Ready → Draining → Stopped
↓ ↓
Blocked Crashed → Recovering → Ready oder Blocked
```

8. Trust Boundaries und Attack Surface

Die wichtigsten Grenzen liegen zwischen Untrusted Client und HTTP Parser, HTTP Metadata und authentisierter Semantik, ODRE PQC und Protected Handler, Python und Native Loader, Path String und Filesystem Object sowie Worker State und Persistent State.

Attack Surface	Repräsentativer Angriff	Defense Location
Admission	Malformed, Oversized, Smuggling Shape	Admission Gate
Session	Expired oder Cross-session Reuse	Session Engine
Sequence	Duplicate, Rollback, Gap, Race	Atomic Sequence Gate
Request Meaning	Method, Path, Query, Header Substitution	Semantic Binding
Response Meaning	Status, Header, Cookie Substitution	Response Binding
Native Files	TOCTOU und Atomic Replacement	Platform Adapter
Loader	Dependency, Provider, Config Injection	Runtime Trust
Filesystem	Symlink, Junction, Reparse, Unsafe Parent	Object Identity
Lifecycle	Stale Lease, Crash, Early Shutdown	Lifecycle Controller

ODRE PQC ersetzt weder Business Authorization, Application Vulnerability Remediation, Host Hardening, DDoS-Schutz, Physical Security, Data-retention Policy noch vollständige Client Device Integrity. Es schützt den Eintritt eines geschützten Requests in die Anwendung und die dafür notwendige Runtime Trust Chain.

9. Deployment-Architektur

```
PQC-fähiger Client
→ Nginx oder Load Balancer
→ FastAPI + ODRE PQC Boundary
→ Protected Handler und Application Data
↳ Platform-specific Native Runtime
```

Nginx oder Load Balancer behält Routing und Transport. ODRE PQC arbeitet innerhalb von FastAPI vor dem Protected Handler. Multi-instance Designs berücksichtigen Session Affinity oder Shared State, Sequence Consistency, Instance Runtime Identity, Worker Leases, Rolling Restart, Retry-bedingtes Replay und getrennte Public-/Protected-Health-Checks.

Vor Deployment werden Supported OS/Architecture, Wheel-/Runtime-Version und Hash, [doctor](#), reale PQC-[verify](#)-Ergebnisse, Routenpolitik, fehlender Automatic Downgrade, Restart-/Recovery-Verfahren sowie secret-freie Logs und Status geprüft.

10. Produktidentität

Element	Wert
Produkt	ODRE PQC
Version	0.2.9
Official Wheel	odre_pqc-0.2.9-py3-none-any.whl
Wheel SHA-256	A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697
Windows Runtime SHA-256	826303E72A76B10041D26113F465ADD8960825C71F50722D6792174E73F9D12C
Ubuntu Runtime SHA-256	EFAD16698203371D07F25B429F6731B9C0954EEBF5B58A0024789EF0611A78D1
OpenSSL Runtime	3.5.8
PQC	ML-KEM-768 / ML-DSA-65
Protocol	odre-pqc/3
Official OS	Windows Server 2022 AMD64 / Ubuntu 24.04 LTS AMD64

11. Fazit

ODRE PQC definiert die Sicherheitsbedingungen, unter denen ein Application Handler ausgeführt werden darf. Runtime Identity, Handshake, Session, Strict Sequence, Replay Defense, Request/Response Binding und Lifecycle werden als ein Vertrag erzwungen. Kann ein geschützter Request diese Bedingungen nicht beweisen, endet er vor dem Handler.

Der externe Ablauf bleibt [init](#), [doctor](#), [verify](#), [install\(app\)](#) und [status](#); plattformspezifisches Native Trust und kryptografischer Protocol State bleiben intern getrennt. Der Produktwert ist kein weiterer Algorithmusname, sondern eine Application-Embedded Post-Quantum Security Boundary mit erklärbaren, betreibbaren und überprüfbaren Allow-/Block-Bedingungen.